

Assignment II - SDD, BDD, and TDD in AI-Assisted Software Development

Student Information

- Name: Jiang Shuowen (蔣碩文)
 - Student ID: 1113314
 - Course: AI-Assisted Software Development (CS351)
 - Date: 2026/05/29
-

1. Introduction

In the rapidly evolving era of generative AI, AI-assisted software development has become a standard approach for modern engineers. Utilizing Large Language Models (LLMs), developers can generate vast amounts of code within seconds. However, while AI is highly capable, it lacks an intrinsic understanding of specific business logic and development contexts. If a developer provides only vague prompts (e.g., "Help me write a grade calculator"), the resulting AI code often suffers from a lack of architectural alignment, missing edge-case handling, or hidden functional bugs.

Therefore, establishing clear, structured requirements has become more vital than ever in the AI era. Specification-Driven Development (SDD), Behavior-Driven Development (BDD), and Test-Driven Development (TDD) are no longer just traditional software engineering methodologies; they have transformed into core tools used to guide, constrain, and validate AI outputs. Developers must transition from being mere "code writers" to becoming "architects and specification navigators." By utilizing these three approaches to feed AI clear blueprints, interactive behavior scripts, and strict quality gates, we can fully harness the power of AI tools while guaranteeing the correctness and robustness of the software system.

2. Definition of SDD

Specification-Driven Development (SDD) is a development process that centers around a structured "Software Specification Document." Before any coding begins, the development team must explicitly define the system architecture, functional scope, and structural boundaries.

The core elements of SDD include:

- **Goal:** Identifying the fundamental purpose of the software and the problem it aims to solve.
- **Functional Requirements:** Explicitly stating what operations and features the software must perform.
- **Input & Output:** Defining the exact data types, constraints, and formats for incoming data and expected system results.
- **Constraints:** Hard rules, algorithms, or limitations that the system must strictly adhere to.
- **Acceptance Criteria:** Objective conditions used to verify whether a feature is complete and functions as intended.

In the AI era, SDD acts as the baseline for high-quality engineering prompts. It establishes a set of "legal boundaries" for the AI, ensuring the model builds the exact architectural framework required without

hallucinating irrelevant features.

3. SDD: Student Grade Calculator

3.1 Goal

To establish an objective, precise, and fault-tolerant Student Grade Calculator that automatically computes a student's final term score based on four weighted assessment components and maps it onto a designated letter grade to streamline academic record-keeping.

3.2 Functional Requirements

- The system must accept four distinct score components for a student.
- The system must strictly apply the predefined weighting formula to calculate the final term score.
- The system must round the calculated final score to exactly one decimal place before evaluation.
- The system must automatically assign and report the correct letter grade (A, B, C, D, or F) based on the rounded final score.
- The system must implement input validation to intercept invalid formats or scores out of the valid range, immediately halting calculation and outputting a descriptive error message.

3.3 Input

- **Assignment Score:** Floating-point number, valid range: 0.0 to 100.0.
- **Midterm Exam Score:** Floating-point number, valid range: 0.0 to 100.0.
- **Final Exam Score:** Floating-point number, valid range: 0.0 to 100.0.
- **Project Score:** Floating-point number, valid range: 0.0 to 100.0.

3.4 Output

- **Final Score:** Floating-point number rounded to one decimal place (e.g., 87.4).
- **Letter Grade:** A single character string indicating the grade tier (A, B, C, D, or F).
- **Error Message:** A descriptive notification displayed when an invalid entry is processed (e.g., "Error: Score out of range (0-100)").

3.5 Grade Rules

The final weighted score is computed via the following mathematical formula:

$$\text{Final Score} = \text{Assignment} \times 0.30 + \text{Midterm} \times 0.20 + \text{Final Exam} \times 0.30 + \text{Project} \times 0.20$$

The letter grade is mapped using the exact numeric intervals below (evaluated *after* rounding the Final Score to one decimal place):

- **A:** $90.0 \leq \text{Final Score} \leq 100.0$
- **B:** $80.0 \leq \text{Final Score} < 90.0$
- **C:** $70.0 \leq \text{Final Score} < 80.0$
- **D:** $60.0 \leq \text{Final Score} < 70.0$
- **F:** $\text{Final Score} < 60.0$

3.6 Acceptance Criteria

- **[Original Criterion 1 - Rounding Precision and Grade Assignment]:** The system must calculate the weighted grade and round it precisely to one decimal place. The letter grade assignment must rely entirely on this rounded value. For instance, if the mathematical calculation yields **79.95**, it must round up to **80.0**, and the assigned letter grade must be reported as **B** rather than **C**.
 - **[Original Criterion 2 - Absolute Boundary Interception]:** If any of the four score inputs falls below **0.0** or exceeds **100.0**, the system must halt all calculations immediately, refuse to apply weights, and return the specific exception prompt: **"Error: Score out of range (0-100)"**.
-

4. Definition of BDD

Behavior-Driven Development (BDD) is an agile methodology that describes software requirements through user interactions and concrete behavioral scenarios. It translates cold, technical specifications from SDD into human-readable user stories.

BDD utilizes a structured domain-specific language called **Gherkin syntax**, which allows project managers, clients, and developers to align on expectations using a simple format:

- **Given:** The initial state or context of the system prior to any interaction.
- **When:** The specific action or trigger executed by a user or an event.
- **Then:** The expected outcome or observable reaction from the system.

In an AI-driven environment, BDD provides rich behavioral context. It effectively functions as an "intent translator" for AI models, guaranteeing that the generated code aligns with human workflows and specific business scenarios.

5. BDD: Student Grade Calculator

Scenario 1: Student excels in project but struggles with the final exam, still achieving a Grade B

Scenario: Student performs well in project but fails final exam, still maintains Grade B

Given the assignment score is 88.0
And the midterm exam score is 82.0
And the final exam score is 69.0
And the project score is 98.0
When the system calculates the final grade
Then the final score should be 83.1
And the letter grade should be B

Scenario 2: System intercepts invalid teacher input exceeding the maximum score limit

Scenario: Teacher accidentally enters a score exceeding the maximum limit

Given the assignment score is 90.0
And the midterm exam score is 105.0
And the final exam score is 85.0
And the project score is 80.0

```
When the system attempts to calculate the final grade
Then the system should refuse to calculate the final score
And the system should return an error message "Error: Score out of range (0-100)"
```

6. Definition of TDD

Test-Driven Development (TDD) is a software engineering technique where test cases are written prior to implementing the functional code. It emphasizes a "test-first" mindset over traditional post-development testing.

TDD operates in a tight iterative cycle known as the Red-Green-Refactor cycle:

1. Red: Write a comprehensive unit test that captures the requirement. The test will initially fail because the underlying feature has not yet been built.
2. Green: Write the minimum amount of functional code necessary to make the test pass successfully.
3. Refactor: Clean up the codebase, eliminate redundancies, and optimize performance while keeping the test suite green.

In the context of AI-assisted development, TDD acts as the ultimate automated referee. No matter how clean or elegant AI-generated code looks on the surface, pushing it through a pre-written TDD harness exposes hidden logical flaws or hallucinations instantly, ensuring code quality.

7. TDD: Student Grade Calculator

Scenario 1: Normal Test Cases

Test Case 1: Standard Average Student Performance (Expected Grade: C)

- **Input**
 - Assignment: 75.0
 - Midterm: 68.0
 - Final Exam: 72.0
 - Project: 80.0
- **Expected Calculation**
 - $\text{Final Score} = 75.0 \times 0.30 + 68.0 \times 0.20 + 72.0 \times 0.30 + 80.0 \times 0.20$
 - $\text{Final Score} = 22.5 + 13.6 + 21.6 + 16.0 = 73.7$
- **Expected Output**
 - Final Score: 73.7
 - Letter Grade: C

Test Case 2: Low-Tier Passing Performance (Expected Grade: D)

- **Input**

- Assignment: 62.0
- Midterm: 58.0
- Final Exam: 60.0
- Project: 65.0
- **Expected Calculation**
 - $\text{Final Score} = 62.0 \times 0.30 + 58.0 \times 0.20 + 60.0 \times 0.30 + 65.0 \times 0.20$
 - $\text{Final Score} = 18.6 + 11.6 + 18.0 + 13.0 = 61.2$
- **Expected Output**
 - Final Score: 61.2
 - Letter Grade: D

Scenario 2: Boundary Test Cases

Test Case 1: Maximum Perfect Score Boundary (Expected Grade: A)

- **Input**
 - Assignment: 100.0
 - Midterm: 100.0
 - Final Exam: 100.0
 - Project: 100.0
- **Expected Calculation**
 - $\text{Final Score} = 100.0 \times 0.30 + 100.0 \times 0.20 + 100.0 \times 0.30 + 100.0 \times 0.20 = 100.0$
- **Expected Output**
 - Final Score: 100.0
 - Letter Grade: A

Test Case 2: Rounding Milestone Boundary (79.95 rounds up to Grade B)

- **Input**
 - Assignment: 80.0
 - Midterm: 79.75
 - Final Exam: 80.0
 - Project: 80.0
- **Expected Calculation**
 - $\text{Final Score} = 80.0 \times 0.30 + 79.75 \times 0.20 + 80.0 \times 0.30 + 80.0 \times 0.20$
 - $\text{Final Score} = 24.0 + 15.95 + 24.0 + 16.0 = 79.95 \rightarrow \text{Rounds to } 80.0$
- **Expected Output**
 - Final Score: 80.0
 - Letter Grade: B

Scenario 3: Invalid Input Test Cases

Test Case 1: Negative Numeric Score Input

- **Input**
 - Assignment: -10.0
 - Midterm: 85.0
 - Final Exam: 90.0
 - Project: 80.0
- **Expected Calculation**
 - The system detects that the Assignment score (-10.0) violates the lower boundary rule (< 0.0) during the verification step. It triggers the exception handler, halting any math execution.
- **Expected Output**
 - Final Score: null
 - Letter Grade: null
 - Error Message: "Error: Score out of range (0-100)"

Test Case 2: Input Value Exceeding the 100-Point Maximum Limit

- **Input**
 - Assignment: 90.0
 - Midterm: 80.0
 - Final Exam: 125.0
 - Project: 85.0
- **Expected Calculation**
 - The validation module catches that the Final Exam input (125.0) surpasses the upper boundary rule (> 100.0), throwing an out-of-range exception immediately.
- **Expected Output**
 - Final Score: null
 - Letter Grade: null
 - Error Message: "Error: Score out of range (0-100)"

8. Comparison of SDD, BDD, and TDD

Item	SDD	BDD	TDD
Full Name	Specification-Driven Development	Behavior-Driven Development	Test-Driven Development
Main Focus	Structural boundaries, functional scopes, and hard algorithmic formulas.	Interactive user workflows, operational scenarios, and story narratives.	Code unit precision, input/output boundary safety, and error handling.
Main Question	"What functional rules and specifications must be established?"	"How should the software behave across various user scenarios?"	"How do we structurally verify that individual pieces of code are correct?"
Typical Format	Technical specifications, hierarchical lists, requirement matrices.	Given-When-Then text scripts written in Gherkin syntax.	Automated unit test scripts and structured parameter tables.

Item	SDD	BDD	TDD
AI-Era Value	Serves as the structural framework for prompts, preventing AI scope creep.	Functions as an intent translator, guiding AI toward accurate user logic.	Acts as an automated referee, identifying bugs and logical hallucinations.
Primary Audience	System Architects, Senior Developers, and Product Managers.	Business Clients, Product Managers, QA Specialists, and Developers.	Software Developers, Automated QA Engineers, and CI/CD pipelines.

9. Reflection

Throughout this assignment, I have come to realize how methodologies like SDD, BDD, and TDD undergo a significant paradigm shift within AI-assisted software development. Traditionally viewed as rigid documentation steps in large enterprises, these practices have transformed into highly efficient tools for managing and steering AI engines.

Among the three methodologies, **BDD is the easiest for me to understand**. Because BDD utilizes the **Given-When-Then** format, it maps perfectly onto human thought processes and natural language. Both technical and non-technical stakeholders can instantly grasp how a system operates under specific conditions. This high readability makes it an intuitive tool when using conversational interfaces to explain complex scenarios to AI.

When working alongside AI tools like GitHub Copilot or ChatGPT, **SDD provides the strongest guiding direction, while TDD is the most valuable for quality control**. SDD drastically minimizes the issues caused by "vague prompts." Simply instructing an AI to "write a grade calculator" often yields inconsistent results, such as missing rounding algorithms or misaligned weightings. However, by supplying an SDD document detailing the exact floating-point inputs, weight structures, and rounding expectations, the accuracy of the initial AI code output improves dramatically.

Nevertheless, AI models frequently overlook edge cases, making **TDD the ultimate safety net**. By designing normal, boundary, and invalid input test suites ahead of time, we can run the AI-generated code against these criteria immediately. If the AI omits an error-handling exception or messes up a decimal placement, the test harness instantly flags a red light. This prompts immediate refactoring until all tests turn green, ensuring the code is production-ready.

In my future software engineering projects, I intend to implement a **combined engineering workflow**:

- 1. Define the blueprint via SDD:** Establish the structural specifications, algorithmic constraints, and limits as a foundational prompt for the AI.
- 2. Elaborate behaviors using BDD:** Map out complex interaction workflows through **Given-When-Then** user stories so the AI captures user expectations.
- 3. Enforce validation with TDD:** Prompt the AI to build unit test cases based on my designed boundary and invalid datasets first, then write the functional code to satisfy those tests until they pass.

10. References / AI Tool Usage

- Gemini (2026):** Utilized to clarify the architectural roles of SDD, BDD, and TDD within automated coding pipelines and to format the Markdown report structure.

- **Cucumber Gherkin Syntax Reference:** Consulted for standard practices regarding the integration of **Given-When-Then** structures.
- All numerical weight configurations, original Gherkin behaviors, and six standalone TDD test scenarios within this Student Grade Calculator report were designed and calculated independently to ensure originality.